

Report tesi

Secondo report di avanzamento tesi - aggiornamento al 2 agosto 2026

1. Obiettivo e perimetro del lavoro

Questo report ricostruisce il lavoro svolto dopo il report di luglio 2026. Il punto di arrivo della fase precedente era una pipeline MQT Predictor compresa e riproducibile; il punto di arrivo di questa fase è un insieme di artefatti più robusto, un training set supervisionato, un primo dataset per LLM con vincoli espliciti di provenienza e validazione e, infine, lo scheletro del prototipo che utilizzerà l'LLM.

2. Stato iniziale: da dove sono partito

Il report precedente aveva chiarito la separazione centrale di MQT Predictor 2025: un classificatore supervisionato seleziona l'hardware, poi una policy RL specifica per la coppia hardware e figure of merit sceglie la sequenza dei pass di compilazione.

Il prossimo step da qui era quello di pensare a come creare un dataset che potesse essere utile all'LLM.

Ricordando che l'obiettivo del modello è quello di consigliare buone opzioni di compilazione, dato in input un circuito quantistico qualsiasi, è assolutamente necessario che il modello abbia un riferimento su che cosa sia una buona risposta.

Oltre alla costruzione del dataset, il prossimo step era quello di pensare allo scheletro dell'assistente/prototipo che utilizzerà l'LLM. Questo perché, ovviamente, il modello costituisce soltanto il nucleo del sistema e, per poterlo utilizzare efficacemente, è necessario costruire attorno a esso un'adeguata infrastruttura.

3. Che cosa ho deciso di fare

3.1 Come strutturare il dataset

All'inizio avevo pensato di strutturare il dataset per l'LLM come un training set etichettato, cioè un insieme di coppie circuito × parametri consigliati giusti. Poi, però, ho notato che un dataset del genere poteva essere troppo poco esaustivo per rendere l'LLM capace di dare una spiegazione finale all'utente sul perché fossero stati consigliati proprio quei parametri.

A questo fine ho deciso di includere, per ogni circuito, l'intera storia della sua compilazione.

Ripercorrendo la pipeline di MQT Predictor, per ogni circuito vengono prima estratti il feature vector e le informazioni di base. Il modello ML seleziona quindi l'hardware più adatto, mentre il relativo modello RL sceglie ed esegue i passi di compilazione. Queste sono tutte informazioni importantissime per l'LLM

Per spiegare un'altra riflessione, ritengo necessario fare un breve riepilogo di come funziona l'addestramento del classificatore supervisionato (ML) in MQT Predictor:

Prima di poter addestrare il modello e quindi fare il fitting, c'è bisogno di avere un training set etichettato, in questo caso composto da coppie di circuiti × hardware migliore. Questo training set non è disponibile a priori, ma deve essere generato compilando ogni circuito con tutti i modelli RL considerati

Questo perché, per ogni circuito x , se abbiamo tre modelli RL A, B e C, ognuno di questi modelli proverà a compilare x e ognuno di loro riceverà uno score che dipende dalla figure of merit scelta. Verrà quindi scelto come miglior hardware del circuito x solo il migliore (cioè chi ha lo score più alto) tra A, B e C.

Questa digressione mi è servita per spiegarvi un'altra scelta che ho fatto per costruire il dataset dell'LLM.

Infatti, da questo possiamo capire che, anche se solo uno tra i modelli è il migliore, non significa che gli altri non siano accettabili e vadano scartati. Ho quindi pensato di salvare le informazioni di tutti i modelli RL nel dataset per l'LLM, non solo le informazioni dei modelli vincitori. In questo modo il modello LLM avrà più scelta e, anche se magari perderà leggermente in performance, avrà più possibilità di restituire una risposta valida, perché conosce più esempi validi!

Inoltre, questa scelta è dettata anche dai limiti hardware con cui sto sviluppando la tesi. Infatti, i modelli RL sono piccoli (anche se pesanti in termini di memoria) e molto spesso falliscono le compilazioni. Quindi, ogni compilazione che abbia successo, anche se non è la migliore, è un'ottima informazione da aggiungere al dataset del modello LLM.

Quindi, per raggiungere questo obiettivo, ho innanzitutto riallenato 5 modelli RL per renderli leggermente più capaci di concludere una compilazione con successo (in particolare, ho allenato `ibm_falcon_27`, `ibm_falcon_127`, `ibm_heron_133`, `ibm_heron_156` e `quantinuum_h2_56`); poi ho tentato di allenare il modello ML su questi 5 hardware con metrica `expected_fidelity`. Come spiegato prima, essendo i modelli RL piccoli, sui 600 circuiti presenti per il training solo pochi sono riusciti a essere compilati; quindi, per ora, il dataset è piccolo, ma non per questo di bassa qualità.

3.2 Architettura prototipo

Il prototipo ha il compito di fare da infrastruttura attorno all'LLM.

Ho pensato a una struttura modulare, in modo da assegnare a ogni parte un singolo compito.

La struttura generale parte da una UI che interagisce con l'utente e riceve le sue richieste testuali.

Ricevuti in input il circuito logico e altri eventuali vincoli testuali voluti dall'utente, questi verranno passati al modulo di parsing, che validerà l'input dell'utente e ne estrarrà le caratteristiche del circuito e i vincoli espressi dall'utente (ad esempio, l'esclusione a priori di alcuni backend).

Una volta avvenuto il parsing dell'input, entra in gioco il filtro di compatibilità, il cui compito è applicare una maschera all'input, rimuovendo tutto ciò che non rispetta i vincoli tecnici del circuito e i vincoli espressi dall'utente; in questo modo si ottiene un input pulito per l'LLM, riducendo al minimo gli elementi non validi.

Arrivati a questo punto, ci sarà un modulo il cui compito sarà quello di costruire un prompt compatto, pronto a essere inviato all'LLM. Questo prompt verrà aumentato attraverso l'aggiunta di esempi prelevati dal dataset (in pratica, un sistema RAG).

Una volta definito il prompt, viene chiamato il Large Language Model, che ci restituirà una risposta.

Questa risposta andrà validata nuovamente, come si è fatto per l'input; infatti, il modello potrebbe allucinare e fornire risposte non valide nonostante tutte le operazioni eseguite in precedenza per creare l'input più pulito possibile.

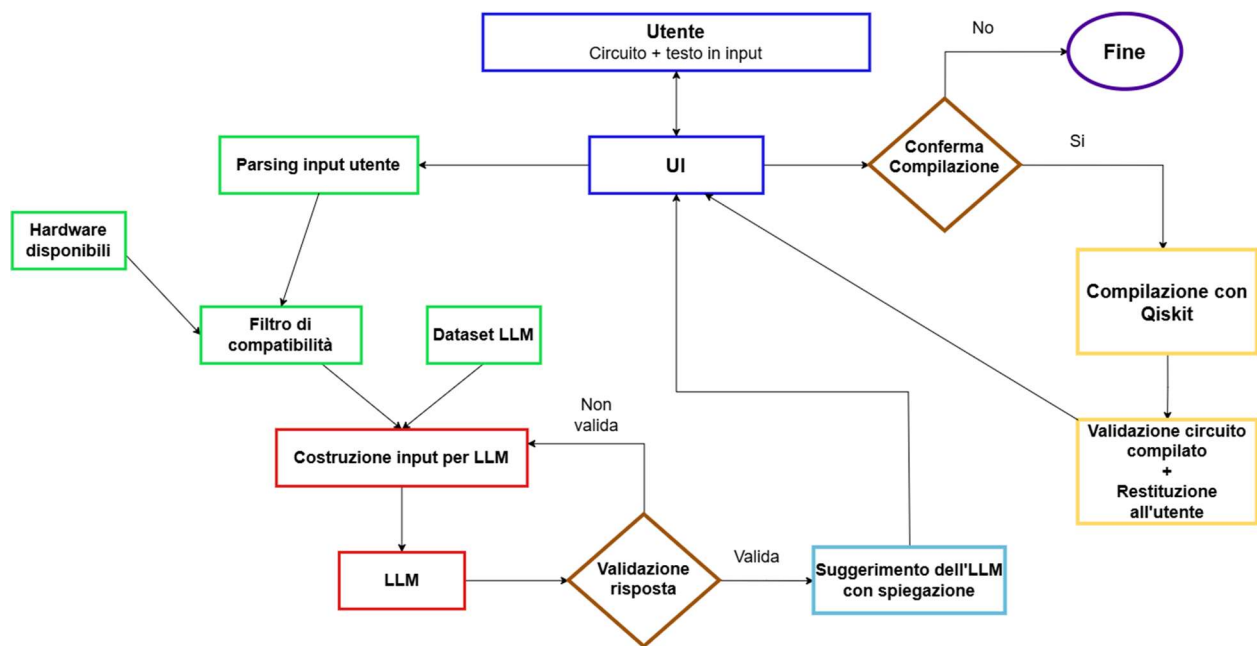
Ho pensato fosse giusto, in caso di invalidità della risposta, lasciar ritentare il modello per un numero massimo di volte ancora da definire, aumentando il prompt con l'errore commesso (è da valutare se possa essere utile o meno).

Se invece la risposta è valida, viene restituita all'utente, che a quel punto può decidere di fermarsi o di chiedere al prototipo di lanciare la compilazione.

Quest'ultima sarà rigorosamente effettuata da compilatori deterministici, come Qiskit. Per ovvi motivi, non andrebbe incaricato il modello di questo difficile compito.

Il circuito compilato andrà infine validato un'ultima volta per vedere se le scelte dell'LLM si sono rivelate adatte.

Di seguito è riportato un diagramma della struttura del prototipo.



4. Quali test ho eseguito

Questa è stata più una fase di costruzione che di test; infatti, i cambiamenti dall'ultima versione del codice sono principalmente questi:

- 1) Riorganizzazione del workspace: l'ambiente stava diventando confusionario; perciò, ho riorganizzato il lavoro in poche cartelle necessarie.
- 2) Modifica dello script di training dei modelli ML: ho modificato lo script per renderlo resistente agli errori, rendendolo capace di terminare o di essere interrotto correttamente. Questo è stato necessario perché molto spesso i modelli RL non riuscivano a compilare con successo i circuiti e ciò mandava in difficoltà il training. In questa versione, invece, se nessuno riesce a compilare il circuito, allora quel circuito viene saltato. Ciò comporta una riduzione del Training set e, di conseguenza, del Dataset destinato all'LLM. Al momento considero questa riduzione un compromesso accettabile rispetto alle risorse hardware disponibili.
- 3) Scrittura di uno script per creare il dataset LLM in formato JSON a partire dal training set generato dall'allenamento del modello ML (script 04). Il file JSON è estremamente lungo, ma strutturato; la sua struttura è spiegata meglio nei README del progetto.
- 4) Implementazione dello scheletro del prototipo: dentro la cartella prototype/ c'è il README che spiega bene la forma del prototipo. (Ovviamente è uno scheletro, non ancora pronto per essere eseguito, visto che bisogna ancora scegliere quale LLM usare.)

5. Quali risultati ho ottenuto

5.1 Datasets

In datasets/ abbiamo 2 file JSON: uno è "device_selector_expected_fidelity.json", che è il training set del modello ML; l'altro, invece, è il dataset JSON per l'LLM.

5.2 Struttura del prototipo LLM

In prototype/ c'è l'intera struttura del prototipo, che rispetta quanto descritto nella sezione 3.2.

5.3 Modelli RL più grandi

Ho provato ad addestrare i 5 modelli RL menzionati in fondo alla sezione 3.1 fino a 8192 step, addestrandoli più a lungo rispetto ai modelli precedenti, che si fermavano a 2048 step. I modelli risultano ancora piccoli; infatti, si riesce a compilare una piccola percentuale dei circuiti presenti nel training set del modello ML.

Continuerò ad allenarli cercando di arrivare a un risultato accettabile per ottenere un dataset migliore.

5.4 Risultati negativi

I tentativi su hardware ulteriori hanno mostrato limiti concreti. Rigetti Ankaa 84 fallisce perché il gate nativo rxpi non è supportato nella conversione BQSKit usata; IonQ Aria 25 fallisce perché BasisTranslator non trova una traduzione da u3/cx alla base gpi/gpi2/ms; il tentativo IQM ha lasciato un log, ma nessun modello o checkpoint finale. In generale, non riesco a usare molti hardware presenti poiché ci sono problemi con la traduzione dei gate e la scelta di passi di compilazione di BQSKit non validi, che non permettono l'addestramento regolare dei relativi modelli. Questi sono problemi che potrebbero essere risolti nelle successive versioni di MQT Predictor.

6.Domande e prossimi steps

Arrivati a questo punto, ho un po' di domande:

- 1) Quale criterio mi consigliate di usare per scegliere il modello LLM adatto?
- 2) La struttura del prototipo va bene o si possono fare miglioramenti? Se sì, quali?
- 3) Le idee per la costruzione del dataset vi sembrano corrette? Avete ulteriori consigli per poter migliorare il dataset? Essendo quest'ultimo forse la parte più importante affinché la qualità del modello sia buona, vorrei essere sicuro di essere sulla strada giusta.
- 4) Per valutare il prototipo, utilizzerei un insieme di circuiti non presenti nei dati precedenti (circuiti nuovi di zecca). Ogni circuito verrebbe compilato con Qiskit utilizzando i parametri suggeriti dall'LLM. Calcolerei quindi lo score del risultato mediante le funzioni di MQT Predictor e confronterei tale valore con quello ottenuto compilando lo stesso circuito tramite MQT Predictor. Ripeterei infine il confronto usando una compilazione Qiskit standard come ulteriore riferimento.
Ditemi se vi sembra una buona idea o se è troppo semplificativa. In base alle risposte a queste domande, i miei prossimi step potrebbero cambiare, ma idealmente, se siete d'accordo su tutto, direi che i prossimi step sono questi:

- 1) Scegliere il modello LLM, installarlo e completare il prototipo.

2) Cercare di implementare un RAG che peschi dal dataset le informazioni necessarie (non so quanto possa essere dispendioso o difficoltoso farlo; in tal caso, i vostri consigli sono ben accetti).

3) Provare ad aumentare la dimensione dei modelli RL per avere più compilazioni riuscite e quindi, di conseguenza, un dataset migliore.

4) Iniziare a creare un ambiente strutturato e organizzato per la fase di sperimentazione (quindi scegliere i circuiti, hardware e figure of merit con cui comparare il prototipo e la baseline)

Riferimenti

[1] N. Quetschlich, L. Burgholzer, R. Wille, MQT Predictor: Automatic Device Selection with Device-Specific Circuit Compilation for Quantum Computing, ACM Transactions on Quantum Computing, 2025.

[2] N. Quetschlich, L. Burgholzer, R. Wille, Predicting Good Quantum Circuit Compilation Options, 2023.

[3] Munich Quantum Toolkit, MQT Predictor 2.3.0: documentazione e codice sorgente analizzati nell'ambiente di lavoro.